

Лабораторная работа №7

Быстрая сортировка, сортировка слиянием, пирамидальная сортировка

Цель работы:

1. Реализовать три эффективных алгоритма сортировки: быструю сортировку (Quick Sort), сортировку слиянием (Merge Sort) и пирамидальную сортировку (Heap Sort)
2. Освоить принципы «разделяй и властвуй» на примере быстрой сортировки и сортировки слиянием
3. Изучить структуру данных «куча» (heap) для реализации пирамидальной сортировки
4. Проанализировать и сравнить производительность различных алгоритмов сортировки
5. Понять преимущества и недостатки каждого алгоритма в различных сценариях использования

Теоретическое обоснование

1. Сравнение эффективных сортировок

Алгоритм	Лучший случай	Средний случай	Худший случай	Память	Устойчивость
Быстрая сортировка	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Нет
Сортировка слиянием	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Да
Пирамидальная	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	Нет

2. Быстрая сортировка (Quick Sort)

Принцип работы (разделяй и властвуй):

Выбрать опорный элемент (pivot)

Разделить массив: элементы меньше pivot - слева, больше - справа

Рекурсивно применить к левой и правой частям

```
int partition(int arr[], int low, int high) {  
    int pivot = arr[high]; // опорный элемент (последний)
```

```

int i = low - 1;    // индекс для меньших элементов

for (int j = low; j < high; j++) {
    if (arr[j] <= pivot) {
        i++;
        swap(arr[i], arr[j]);
    }
}

swap(arr[i + 1], arr[high]);
return i + 1; // позиция опорного элемента
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

```

Выбор опорного элемента:

Стратегия	Преимущества	Недостатки
Первый элемент	Просто	Плохо на почти отсортированных
Последний элемент	Просто	Плохо на почти отсортированных
Средний элемент	Лучше	Не гарантирует
Случайный элемент	Хорошо против худшего случая	Требует генерации
Медиана из трёх	Хорошо	Чуть сложнее

3. Сортировка слиянием (Merge Sort)

Принцип работы:

Разделить массив пополам

Рекурсивно отсортировать каждую половину

Слить две отсортированные половины

```

void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int* L = new int[n1];
    int* R = new int[n2];

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];

    delete[] L;
    delete[] R;
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

```

4. Пирамидальная сортировка (Heap Sort)

Принцип работы:

Построить двоичную кучу (max-heap) из массива

Извлекать корень (максимальный элемент) и помещать в конец массива

Восстанавливать свойство кучи

```

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

```

```

    if (left < n && arr[left] > arr[largest])
        largest = left;
    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    // Построение кучи
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Извлечение элементов
    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

```

Исходный массив: [4, 10, 3, 5, 1]

Куча (max-heap): [10, 5, 3, 4, 1]

```

    10
   / \
  5  3
 / \
4  1

```

Примеры выполнения практической части

Пример 1. Быстрая сортировка с разными вариантами выбора опорного элемента

```

#include <iostream>
using namespace std;

// Выбор опорного элемента: последний
int partitionLast(int arr[], int low, int high) {

```

```

int pivot = arr[high];
int i = low - 1;
for (int j = low; j < high; j++) {
    if (arr[j] <= pivot) {
        i++;
        swap(arr[i], arr[j]);
    }
}
swap(arr[i + 1], arr[high]);
return i + 1;
}

```

// Выбор опорного элемента: медиана из трёх

```

int medianOfThree(int arr[], int low, int high) {
    int mid = low + (high - low) / 2;
    if (arr[low] > arr[mid]) swap(arr[low], arr[mid]);
    if (arr[low] > arr[high]) swap(arr[low], arr[high]);
    if (arr[mid] > arr[high]) swap(arr[mid], arr[high]);
    swap(arr[mid], arr[high]);
    return arr[high];
}

```

```

int partitionMedian(int arr[], int low, int high) {
    medianOfThree(arr, low, high);
    return partitionLast(arr, low, high);
}

```

```

void quickSort(int arr[], int low, int high, int (*partitionFunc)(int[], int, int))
{
    if (low < high) {
        int pi = partitionFunc(arr, low, high);
        quickSort(arr, low, pi - 1, partitionFunc);
        quickSort(arr, pi + 1, high, partitionFunc);
    }
}

```

```

int main() {
    int arr1[] = {64, 34, 25, 12, 22, 11, 90, 5};
    int n = sizeof(arr1) / sizeof(arr1[0]);

    quickSort(arr1, 0, n - 1, partitionLast);
}

```

```

    cout << "Быстрая сортировка (опорный - последний): ";
    for (int x : arr1) cout << x << " ";
    cout << endl;

    return 0;
}

```

Пример 2. Сравнение всех трёх сортировок

```

#include <iostream>
#include <chrono>
#include <algorithm>

using namespace std;
using namespace chrono;

// Быстрая сортировка
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) swap(arr[++i], arr[j]);
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// Сортировка слиянием
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1, n2 = right - mid;
    int* L = new int[n1];
    int* R = new int[n2];

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
}

```

```

    int i = 0, j = 0, k = left
    while (i < n1 && j < n2) arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];

    delete[] L;
    delete[] R;
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

// Пирамидальная сортировка
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest]) largest = left;
    if (right < n && arr[right] > arr[largest]) largest = right;

    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

```

```

long long measureTime(void (*sortFunc)(int[], int), int arr[], int n) {
    int* copy = new int[n];
    for (int i = 0; i < n; i++) copy[i] = arr[i];

    auto start = high_resolution_clock::now();
    sortFunc(copy, n);
    auto end = high_resolution_clock::now();

    delete[] copy;
    return duration_cast<microseconds>(end - start).count();
}

int main() {
    const int N = 50000;
    int* arr = new int[N];

    for (int i = 0; i < N; i++) arr[i] = rand() % N;

    cout << "Сравнение производительности (N = " << N << "):" << endl;

    // Для быстрой сортировки нужна обёртка
    int* copy = new int[N];
    for (int i = 0; i < N; i++) copy[i] = arr[i];
    auto start = high_resolution_clock::now();
    quickSort(copy, 0, N - 1);
    auto end = high_resolution_clock::now();
    cout << "Быстрая сортировка: " << duration_cast<microseconds>(end -
start).count() << " мкс" << endl;
    delete[] copy;

    cout << "Сортировка слиянием: " << measureTime([&int a[], int n) {
mergeSort(a, 0, n - 1); }, arr, N) << " мкс" << endl;
    cout << "Пирамидальная:      " << measureTime(heapSort, arr, N) << "
мкс" << endl;

    delete[] arr;
    return 0;
}

```

Пример 3. Сортировка слиянием для больших файлов

```
#include <iostream>
```

```

#include <fstream>
#include <vector>
#include <algorithm>

using namespace std;

void merge(vector<int>& arr, int left, int mid, int right) {
    vector<int> temp(right - left + 1);
    int i = left, j = mid + 1, k = 0;

    while (i <= mid && j <= right) {
        temp[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];
    }
    while (i <= mid) temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];

    for (int idx = 0; idx < k; idx++) arr[left + idx] = temp[idx];
}

void mergeSort(vector<int>& arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
    vector<int> numbers;
    ifstream in("numbers.txt");
    int x;
    while (in >> x) numbers.push_back(x);
    in.close();

    mergeSort(numbers, 0, numbers.size() - 1);

    ofstream out("sorted.txt");
    for (int num : numbers) out << num << " ";
    out.close();

    cout << "Сортировка завершена. Результат в sorted.txt" << endl;
}

```

```

return 0;
}

```

Индивидуальные задания

Базовый уровень (15 вариантов)

Общее задание: Реализовать указанные алгоритмы сортировки и протестировать их на массивах разного размера.

№ вар.	Алгоритмы	Тестовые данные
1	Быстрая сортировка	Случайный массив, размеры 100, 1000, 10000
2	Сортировка слиянием	Случайный массив, размеры 100, 1000, 10000
3	Пирамидальная сортировка	Случайный массив, размеры 100, 1000, 10000
4	Быстрая + слиянием	Сравнить на случайных массивах
5	Быстрая + пирамидальная	Сравнить на случайных массивах
6	Слиянием + пирамидальная	Сравнить на случайных массивах
7	Быстрая сортировка	Почти отсортированный массив
8	Сортировка слиянием	Почти отсортированный массив
9	Пирамидальная сортировка	Почти отсортированный массив
10	Все три сортировки	Сравнить на случайных массивах (N = 10000)
11	Все три сортировки	Сравнить на почти отсортированных массивах
12	Все три сортировки	Сравнить на обратно отсортированных массивах

№ вар.	Алгоритмы	Тестовые данные
13	Быстрая сортировка	Оптимизировать выбор опорного элемента (медиана трёх)
14	Сортировка слиянием	Реализовать без использования дополнительной памяти
15	Пирамидальная сортировка	Реализовать итеративную версию heapify

Средний уровень (15 вариантов)

Общее задание: Реализовать указанные алгоритмы сортировки с дополнительными требованиями и провести детальный анализ.

№ вар.	Алгоритмы	Дополнительные требования
1	Быстрая сортировка	Реализовать 3 варианта выбора опорного элемента и сравнить
2	Сортировка слиянием	Реализовать сортировку для связного списка
3	Пирамидальная сортировка	Реализовать min-heap (сортировка по убыванию)
4	Быстрая + слиянием	Подсчитать количество сравнений и обменов для каждого
5	Быстрая + пирамидальная	Подсчитать количество сравнений и обменов для каждого
6	Слиянием + пирамидальная	Подсчитать количество сравнений и обменов для каждого
7	Быстрая сортировка	Реализовать итеративную версию (без рекурсии)

№ вар.	Алгоритмы	Дополнительные требования
8	Сортировка слиянием	Реализовать сортировку для строк (лексикографически)
9	Пирамидальная сортировка	Реализовать сортировку для структуры Student (по среднему баллу)
10	Все три сортировки	Построить график зависимости времени от N (N = 1000, 2000, 5000, 10000)
11	Все три сортировки	Протестировать на различных типах данных (int, double, string)
12	Быстрая сортировка	Реализовать 3-х частное разделение (для повторяющихся элементов)
13	Сортировка слиянием	Реализовать естественную сортировку слиянием (используя уже упорядоченные подмассивы)
14	Пирамидальная сортировка	Реализовать сортировку на месте без рекурсии
15	Все три сортировки	Сравнить с std::sort и проанализировать разницу

Продвинутый уровень (15 вариантов)

Общее задание: Реализовать все три сортировки с полной функциональностью и дополнительными возможностями. Провести углублённый анализ.

№ вар.	Дополнительные требования
1	Реализовать шаблонные версии всех сортировок для любых типов
2	Реализовать сортировку слиянием для внешней сортировки (файлы)

№ вар.	Дополнительные требования
3	Реализовать параллельную версию быстрой сортировки (OpenMP)
4	Реализовать сортировку слиянием для потоков (Stream API)
5	Реализовать стабильную версию быстрой сортировки
6	Реализовать интроспективную сортировку (гибрид быстрой + пирамидальной)
7	Реализовать блочную сортировку слиянием (TimSort)
8	Реализовать сортировку слиянием для больших файлов (>RAM)
9	Реализовать итеративную версию быстрой сортировки с ручным стеком
10	Реализовать визуализацию процесса сортировки (графическая)
11	Реализовать профилирование и оптимизацию (кэш-промахи, ветвления)
12	Реализовать сортировку слиянием для распределённых систем
13	Реализовать гибридный алгоритм (быстрая + слиянием + вставками для малых массивов)
14	Реализовать сортировку слиянием для анализа больших данных (MapReduce)
15	Реализовать интерактивную демонстрацию работы каждого алгоритма

Контрольные вопросы

1. Какова сложность быстрой сортировки в лучшем, среднем и худшем случае?
2. Когда возникает худший случай для быстрой сортировки?
3. Как можно улучшить быструю сортировку для защиты от худшего случая?
4. Какова сложность сортировки слиянием? Почему она всегда $O(n \log n)$?

5. Какие преимущества и недостатки имеет сортировка слиянием перед быстрой?
6. Как работает пирамидальная сортировка? Какова её сложность?
7. Какая из трёх сортировок устойчивая? Почему?
8. Какая из трёх сортировок требует дополнительной памяти? Сколько?
9. Какая из трёх сортировок лучше работает на почти отсортированных данных?
10. Зачем в пирамидальной сортировке сначала строится куча?
11. Что такое «разделяй и властвуй»? Как он применяется в быстрой сортировке и сортировке слиянием?
12. Можно ли реализовать быструю сортировку итеративно (без рекурсии)?
13. Почему быстрая сортировка обычно быстрее сортировки слиянием на практике?
14. Когда выгодно использовать пирамидальную сортировку?
15. Какой алгоритм сортировки вы бы выбрали для встроенных систем с ограниченной памятью?

Требования к отчёту

Титульный лист

Цель работы

Теоретическая часть (алгоритмы сортировки, их сложность)

Постановка задачи (согласно варианту)

Листинг программы с комментариями

Результаты работы (скриншоты вывода, таблицы времени выполнения)

Анализ сложности для каждого алгоритма

Сравнительная таблица производительности

Ответы на контрольные вопросы

Выводы

Ссылка на Git-репозиторий

Критерии оценивания

Максимальный балл: 100

№	Критерий	Баллы
1	Быстрая сортировка	25
1.1	Реализация алгоритма	10

№	Критерий	Баллы
1.2	Корректность работы на разных данных	10
1.3	Выбор опорного элемента (оптимизация)	5
2	Сортировка слиянием	25
2.1	Реализация алгоритма	10
2.2	Корректность слияния	10
2.3	Управление памятью (нет утечек)	5
3	Пирамидальная сортировка	25
3.1	Реализация heapify	10
3.2	Построение кучи и сортировка	10
3.3	Корректность работы	5
4	Анализ и сравнение	15
4.1	Измерение времени выполнения	5
4.2	Сравнительная таблица/график	5
4.3	Выводы о производительности	5
5	Оформление и отчёт	10
5.1	Код отформатирован, есть комментарии	5
5.2	Есть ответы на вопросы, ссылка на Git	5
Итого		100

Штрафные баллы

Нарушение	Штраф
Неверная сложность алгоритма	-10
Утечка памяти (в сортировке слиянием)	-10
Нет анализа производительности	-10
Нет ссылки на Git-репозиторий	-10
Программа не компилируется	работа не принимается

Итоговая шкала

Оценка	Баллы
Отлично	90–100
Хорошо	75–89
Удовлетворительно	60–74
Неудовлетворительно	менее 60